

Planner Agent – Developer Guide

This guide explains how to use the `/plan` agent within the MiDAS GitHub workflow to create structured, actionable implementation plans without modifying code.

Introduction

The `/plan` agent is a software architecture and project planning expert integrated into the MiDAS GitHub workflow. It is triggered through GitHub issue comments and produces comprehensive, execution-ready planning documentation.

The agent analyzes scope, requirements, dependencies, risks, and technical constraints to create a phased implementation plan. It does not generate or modify code. Instead, it provides structured planning guidance to help teams execute systematically and predictably.

The purpose of the `/plan` agent is to improve execution clarity, reduce ambiguity, and ensure alignment before implementation begins.

What Are the Use Cases for the Plan Agent

The `/plan` agent should be used when a structured implementation roadmap is required before coding starts.

Common Use Cases

- Breaking down large features into phases
- Planning cross-subsystem integrations
- Preparing execution roadmap for new services
- Defining dependencies before implementation
- Identifying risks and mitigation strategies



- Estimating effort and sequencing work

When to Use `/plan`

Use `/plan` when:

- Architecture is already known or partially defined.
- You need a structured execution roadmap.
- Work needs to be broken into logical milestones.
- Dependencies and risks must be documented.
- The goal is planning, not architecture creation.

Do not use `/plan` to generate code or detailed architecture diagrams. Use `/design` for architecture and `/implement` for code generation.

How to Use the Plan Agent

The `/plan` agent is triggered from GitHub issue comments.

Step 1: Navigate to the GitHub Issue

The GitHub issue should include:

- Objective
- Scope boundaries
- Known constraints
- Expected outcome
- Subsystems involved (if applicable)



Test Issue - Add Health Check Endpoint to SS6 Service #3768

Open KES1HYD opened this issue 7 minutes ago · 0 comments



KES1HYD commented 7 minutes ago

Description

We need to add a basic health check endpoint to the SS6 service. This endpoint should allow external systems and monitoring tools to verify that the service is running and responsive.

The endpoint should return a simple success response when the service is healthy.

The goal of this task is to design the endpoint structure, response format, and integration approach within the existing SS6 architecture.

Acceptance Criteria

- A `/health` or `/healthz` endpoint is defined.
- The endpoint returns HTTP 200 when the service is healthy.
- The response format is clearly defined (JSON).
- The design specifies where the endpoint will be implemented (router/controller).
- Any dependency checks (optional DB, cache, etc.) are defined.
- Security considerations are documented (public vs internal access).

Tasks

- Define API contract for health endpoint.
- Identify integration location in SS6 service.
- Define response structure.
- Document assumptions and constraints.

Step 2: Trigger the Agent

In the issue comment section, type:

```
@MiDAS /plan <your query>
```

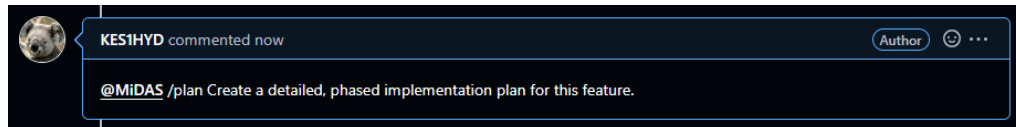
Example:

```
@MiDAS /plan Create a detailed implementation plan for adding a health check endpoint to SS6.
```

Another example:

```
@MiDAS /plan Break this feature into execution phases with risks and dependencies.
```





Step 3: Review the Generated Plan

The agent will respond with a structured planning document that may include:

- Scope definition
- Assumptions
- Prerequisites
- Phase-wise breakdown
- Dependency mapping
- Risk assessment
- Testing strategy
- Deployment considerations
- Success criteria
- Estimated effort or complexity assessment

The `/plan` agent may generate a structured document including:

1. Scope Definition

- Define endpoint behavior
- Define integration boundaries

2. Phase 1 – API Definition

- Confirm route naming
- Define response schema

3. Phase 2 – Implementation

- Add router file
- Register route in main application
- Add logging

4. Phase 3 – Testing



- Unit test endpoint
- Verify HTTP status
- Validate response format

5. Phase 4 – Deployment

- Validate in staging
- Confirm monitoring integration

Risk Assessment

- Incorrect route registration
- Missing dependency health checks

Success Criteria

- Endpoint returns 200
- Passes CI
- Verified in staging



github-actions (bot) commented 4 minutes ago • edited

😊 ...

MiDAS finished @GYH2KOR's task in 2m 39s — [View job](#)

Implementation Plan: Health Check Endpoint for SS6 Service

Overview

This plan details the implementation of a health check endpoint for the SS6 Service API Platform. The health check endpoint will follow existing MiDAS platform patterns observed in SS1, SS7, and SS11 subsystems, providing a simple, unauthenticated endpoint to verify service availability for monitoring and orchestration purposes.

Service Context:

- **Service:** SS6 Service API Platform (`ss6-serviceapi-platform`)
- **Technology Stack:** FastAPI (Python), Uvicorn
- **Current Status:** No health check endpoint exists
- **URL Pattern:** `/services/api/v1/health`

Prerequisites

- ✓ SS6 service located and analyzed (FastAPI-based microservice)
- ✓ Existing health check patterns identified from SS1, SS7, SS11
- ✓ Project structure understood (router-based architecture)
- ✓ Testing framework identified (pytest with pytest-asyncio)
- ✓ Dependencies confirmed (FastAPI 0.115.8, pytest 8.4.1)

Required Access:

- Write access to SS6 codebase
- Ability to run local tests
- Access to SS6 deployment configuration (for future deployment)

Implementation Phases

Phase 1: Create Health Check Router



Phase 3: Implement Unit Tests

Complexity: Low

Description:

Create comprehensive unit tests for the health check endpoint following the existing SS6 testing patterns.

Steps:

1. Create test file structure

```
ss6-serviceapi-platform/tests/unit/api/health_management/  
├── __init__.py  
└── test_health_router.py
```



2. Implement test_health_router.py

- Import necessary test dependencies (pytest, TestClient, FastAPI)
- Add path resolution for imports (following existing pattern from test_analytics_service.py)
- Create test fixtures for FastAPI TestClient
- Implement test cases:
 - `test_health_check_success()` : Verify 200 status code
 - `test_health_check_response_structure()` : Validate response format
 - `test_health_check_response_content()` : Verify status message
 - `test_health_check_logging()` : Ensure logging occurs (optional)

3. Test implementation pattern

```
@pytest.mark.asyncio  
async def test_health_check_success(test_client):  
    response = test_client.get("/services/api/v1/health")  
    assert response.status_code == 200  
    assert "status" in response.json()
```



File Locations:

- `ss6-serviceapi-platform/tests/unit/api/health_management/test_health_router.py`

Dependencies:

- Phase 1 and 2 completion
- pytest, pytest-asyncio (already in requirements.txt)

Success Criteria:

- ✓ Test file created with proper structure



Success Criteria - Final Checklist

- ☐ Health endpoint accessible at `/services/api/v1/health`
- ☐ Returns HTTP 200 status code
- ☐ Response format matches platform pattern
- ☐ No authentication required
- ☐ Endpoint appears in OpenAPI documentation
- ☐ Unit tests pass with >80% coverage
- ☐ No regression in existing functionality
- ☐ Code review approved
- ☐ Documentation updated
- ☐ Deployment successful across all environments
- ☐ Monitoring tools configured to use new endpoint

References

Related Files Analyzed:

- `ss1-llms/app/api/v1/health_management/health_router.py` - Pattern reference
- `ss7-pmtintegration/app/api/v1/health/health_router.py` - Pattern reference
- `ss11-session-analytics/app/api/v1/health_management/health_router.py` - Pattern reference
- `ss6-serviceapi-platform/app/appmain.py` - Main application file
- `ss6-serviceapi-platform/app/api/v1/analytics/analytics_router.py` - Router pattern example
- `ss6-serviceapi-platform/tests/unit/services/analytics/test_analytics_service.py` - Testing pattern example

Platform Documentation:

- `docs/updated_block_diagrams/ss6_services.md` - SS6 architecture overview

Next Steps

This implementation plan is ready for review and execution. The plan follows established MiDAS platform patterns and provides a clear, phased approach to adding a health check endpoint to the SS6 Service API Platform.

Recommended Action: Review this plan with the team, then proceed with Phase 1 implementation.

What This Validates

Using this example, you can verify that the Plan Agent:

- Breaks work into logical phases
- Identifies dependencies
- Documents risks clearly
- Provides structured execution guidance
- Aligns planning with acceptance criteria



Best Practices

To get high-quality planning outputs:

- Clearly define scope boundaries.
 - Specify subsystems involved.
 - Mention integration dependencies.
 - Clarify whether architecture is already finalized.
 - Indicate constraints (timeline, environment, technology restrictions).
-

Governance and Review

The generated plan should:

- Be reviewed by the responsible engineering lead.
- Be validated against organizational priorities.
- Be aligned with sprint or roadmap objectives.
- Be used as execution reference during implementation.

The `/plan` agent assists structured execution but does not replace project ownership or stakeholder alignment.

Limitations

- The agent does not modify code.
- It does not estimate exact delivery timelines unless context is provided.
- It assumes clarity in scope and requirements.
- It does not replace architectural validation where needed.



Summary

The `/plan` agent enables teams to convert feature requirements into structured, actionable implementation roadmaps directly from GitHub issues.

It ensures clarity before development begins, reduces execution ambiguity, and improves predictability across the organization.

